Lab 6: Introducing Classification

Objectives:

- To gain hands-on experience classifying small dataset
- To implement concepts related to Decision Tree classifier (i.e. Entropy, Information Gain), along with using existing libraries.

In [ ]:
```
# Run this cell if you use Colab
from google.colab import drive
drive.mount('/content/drive')
```

Code it yourself

In [1]:
```
import pandas as pd

# Read the data
df = pd.read_csv('toy_data.csv')
df
```

Out[1]:

	age	income	student	credit rating	buys computer
0	<=30	high	no	fair	no
1	<=30	high	no	excellent	no
2	31-40	high	no	fair	yes
3	>40	medium	no	fair	yes
4	>40	low	yes	fair	yes
5	>40	low	yes	excellent	no
6	31-40	low	yes	excellent	yes
7	<=30	medium	no	fair	no
8	<=30	low	yes	fair	yes
9	>40	medium	yes	fair	yes
10	<=30	medium	yes	excellent	yes
11	31-40	medium	no	excellent	yes
12	31-40	high	yes	fair	yes
13	>40	medium	no	excellent	no

In [2]:
```
print(df.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 14 entries, 0 to 13
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             14 non-null     str
 1   income          14 non-null     str
 2   student         14 non-null     str
 3   credit rating   14 non-null     str
 4   buys computer   14 non-null     str
dtypes: str(5)
memory usage: 692.0 bytes
None
```

TODO: Write functions to compute Gain and Entropy, as discussed in the lecture.

In [6]:
```
df["buys computer"].value_counts().to_dict()
```

Out[6]:
```
{'yes': 9, 'no': 5}
```

In [8]:
```
import math
```

In [ ]:
```
# Write your code here
def calEntropy(col):
    en = 0
    dictJ = col.value_counts().to_dict()
    n = len(col)
    for i in dictJ.values() :
        en -= math.log2(i/n)*(i/n)
    return en

calEntropy(df["age"])
```

Out[ ]:
```
1.5774062828523454
```

In [71]:
```
calEntropy(df["buys computer"])
```

Out[71]:
```
0.9402859586706311
```

In [49]:
```
def calGain(par,t):
    enp =calEntropy(df[t])
    sumeni = 0
    pUq = df[par].unique()
    n = len(df[par])
    for i in pUq :
        dfI = df[df[par]==i][t]
        ni = len(dfI)
        sumeni += calEntropy(dfI)*(ni/n)
    return enp-sumeni

calGain("age","buys computer")
```

Out[49]:
```
0.24674981977443933
```

In [50]:
```
calGain("income","buys computer")
```

Out[50]: 0.02922256565895487

In [51]:
```
calGain("student","buys computer")
```

Out[51]: 0.15183550136234159

In [53]:
```
calGain("credit rating","buys computer")
```

Out[53]: 0.048127030408269 49

Using Libraries

Now that you know how to compute these values by yourselfs, now let's use some libraries.

Steps:

- Split the Data → Divide dataset into training (80%) and testing (20%).
- Train the Model → Fit a Decision Tree using the training data.
- Test the Model → Use the trained model to predict on test data.
- Evaluate Performance → Compare predictions with actual values (e.g., Accuracy Score).

Prepare features and labels.

In [59]:
```
# Features
features = df.drop('buys computer', axis=1)
features

# Alternatively, you can use this:
# features = df.iloc[:, :-1]
```

Out[59]:

	age	income	student	credit rating
0	<=30	high	no	fair
1	<=30	high	no	excellent
2	31-40	high	no	fair
3	>40	medium	no	fair
4	>40	low	yes	fair
5	>40	low	yes	excellent
6	31-40	low	yes	excellent
7	<=30	medium	no	fair
8	<=30	low	yes	fair
9	>40	medium	yes	fair
10	<=30	medium	yes	excellent
11	31-40	medium	no	excellent
12	31-40	high	yes	fair
13	>40	medium	no	excellent

In [60]:
```
# Labels (or Target)
labels = df['buys computer']
labels

# # Alternatively, you can use this:
# labels = df.iloc[:, [-1]]
```

Out[60]:
```
0      no
1      no
2     yes
3     yes
4     yes
5      no
6     yes
7      no
8     yes
9     yes
10    yes
11    yes
12    yes
13     no
Name: buys computer, dtype: str
```

In [61]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree

# 1. Load the dataset
X = features.values  # Features
y = labels.values  # Target labels

# 2. Split the dataset into training (80%) and testing (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Create and train a Decision Tree model with entropy criterion
clf = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
clf.fit(X_train, y_train)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[61], line 15
     13 # 3. Create and train a Decision Tree model with entropy criterion
     14 clf = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
---> 15 clf.fit(X_train, y_train)

File c:\Users\win25\Desktop\Desktop\work\CPE232 Data models\.venv\Lib\site-packages\sklearn\base.py:1336, in _fit_context.<locals>.decorator.<locals>.wrapper(estimator, *args, **kwargs)
   1329     estimator._validate_params()
   1331 with config_context(
   1332     skip_parameter_validation=(
   1333         prefer_skip_nested_validation or global_skip_validation
   1334     )
   1335 ):
-> 1336     return fit_method(estimator, *args, **kwargs)

File c:\Users\win25\Desktop\Desktop\work\CPE232 Data models\.venv\Lib\site-packages\sklearn\tree\_classes.py:1027, in DecisionTreeClassifier.fit(self, X, y, sample_weight, check_input)
    996 @_fit_context(prefer_skip_nested_validation=True)
    997 def fit(self, X, y, sample_weight=None, check_input=True):
    998     """Build a decision tree classifier from the training set (X, y).
    999
   1000     Parameters
   (...)   1024         Fitted estimator.
   1025     """
-> 1027     super()._fit(
   1028         X,
   1029         y,
   1030         sample_weight=sample_weight,
   1031         check_input=check_input,
   1032     )
   1033     return self

File c:\Users\win25\Desktop\Desktop\work\CPE232 Data models\.venv\Lib\site-packages\sklearn\tree\_classes.py:255, in BaseDecisionTree._fit(self, X, y, sample_weight, check_input, missing_values_in_feature_mask)
    251 check_X_params = dict(
    252     dtype=DTYPE, accept_sparse="csc", ensure_all_finite=False
    253 )
    254 check_y_params = dict(ensure_2d=False, dtype=None)
--> 255 X, y = validate_data(
    256     self, X, y, validate_separately=(check_X_params, check_y_params)
    257 )
    259 missing_values_in_feature_mask = (
    260     self._compute_missing_values_in_feature_mask(X)
    261 )
    262 if issparse(X):

File c:\Users\win25\Desktop\Desktop\work\CPE232 Data models\.venv\Lib\site-packages\sklearn\utils\validation.py:2914, in validate_data(_estimator, X, y, reset, validate_separately, skip_check_array, **check_params)
   2912 if "estimator" not in check_X_params:
   2913     check_X_params = {**default_check_params, **check_X_params}
-> 2914 X = check_array(X, input_name=    , **check_X_params)
   2915 if "estimator" not in check_y_params:
   2916     check_y_params = {**default_check_params, **check_y_params}

File c:\Users\win25\Desktop\Desktop\work\CPE232 Data models\.venv\Lib\site-packages\sklearn\utils\validation.py:1022, in check_array(array, accept_sparse, accept_large_sparse, dtype, order, copy, force_writeable, ensure_all_finite, ensure_non_negative, ensure_2d, allow_nd, ensure_min_samples, ensure_min_features, estimator, input_name)
   1020         array = xp.astype(array, dtype, copy=False)
   1021     else:
-> 1022         array = _asarray_with_order(array, order=order, dtype=dtype, xp=xp)
   1023 except ComplexWarning as complex_warning:
   1024     raise ValueError(
   1025         "Complex data not supported\n{}\n".format(array)
   1026     ) from complex_warning

File c:\Users\win25\Desktop\Desktop\work\CPE232 Data models\.venv\Lib\site-packages\sklearn\utils\_array_api.py:878, in _asarray_with_order(array, dtype, order, copy, xp, device)
    876     array = numpy.array(array, order=order, dtype=dtype)
    877 else:
--> 878     array = numpy.asarray(array, order=order, dtype=dtype)
    880 # At this point array is a NumPy ndarray. We convert it to an array
    881 # container that is consistent with the input's namespace.
    882 return xp.asarray(array)

ValueError: could not convert string to float: '31-40'
```

There's an error:

ValueError: could not convert string to float: '31-40'

In [62]:
```
from sklearn.preprocessing import LabelEncoder

# Initialize LabelEncoder
label_encoder = LabelEncoder()

# Apply Label Encoding for all categorical columns
df['age'] = label_encoder.fit_transform(df['age'])
df['income'] = label_encoder.fit_transform(df['income'])
df['student'] = label_encoder.fit_transform(df['student'])
df['credit rating'] = label_encoder.fit_transform(df['credit rating'])
df['buys computer'] = label_encoder.fit_transform(df['buys computer'])

# Display the encoded DataFrame
print(df)
```

```
    age  income  student  credit rating  buys computer
0     1       0        0              1              0
1     1       0        0              0              0
2     0       0        0              1              1
3     2       2        0              1              1
4     2       1        1              1              1
5     2       1        1              0              0
6     0       1        1              0              1
7     1       2        0              1              0
8     1       1        1              1              1
9     2       2        1              1              1
10    1       2        1              0              1
11    0       2        0              0              1
12    0       0        1              1              1
13    2       2        0              0              0
```

Let's check out an updated dataframe.

In [63]:
```
df
```

Out[63]:

	age	income	student	credit rating	buys computer
0	1	0	0	1	0
1	1	0	0	0	0
2	0	0	0	1	1
3	2	2	0	1	1
4	2	1	1	1	1
5	2	1	1	0	0
6	0	1	1	0	1
7	1	2	0	1	0
8	1	1	1	1	1
9	2	2	1	1	1
10	1	2	1	0	1
11	0	2	0	0	1
12	0	0	1	1	1
13	2	2	0	0	0

In [64]:
```
X = df.drop('buys computer', axis=1)  # Features
y = df['buys computer']  # Target
```

Let's continue where we left off!

In [65]:
```
# 2. Split the dataset into training (80%) and testing (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Create and train a Decision Tree model with entropy criterion
clf = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
clf.fit(X_train, y_train)
```

Out[65]:

DecisionTreeClassifier

Parameters

criterion	'entropy'
splitter	'best'
max_depth	3
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	None
random_state	42
max_leaf_nodes	None
min_impurity_decrease	0.0
class_weight	None
ccp_alpha	0.0
monotonic_cst	None

In [66]:
```
print(X_train.shape)
print(X_test.shape)
```

```
(11, 4)
(3, 4)
```

Now we're going build the Decision Tree Classifier

In [67]:
```
from sklearn.tree import DecisionTreeClassifier

# Initialize the Decision Tree classifier
clf = DecisionTreeClassifier(criterion='entropy', random_state=42)  # Using 'entropy' as the criterion

# Train the model
clf.fit(X_train, y_train)

# Predict on the test set
y_pred = clf.predict(X_test)
```

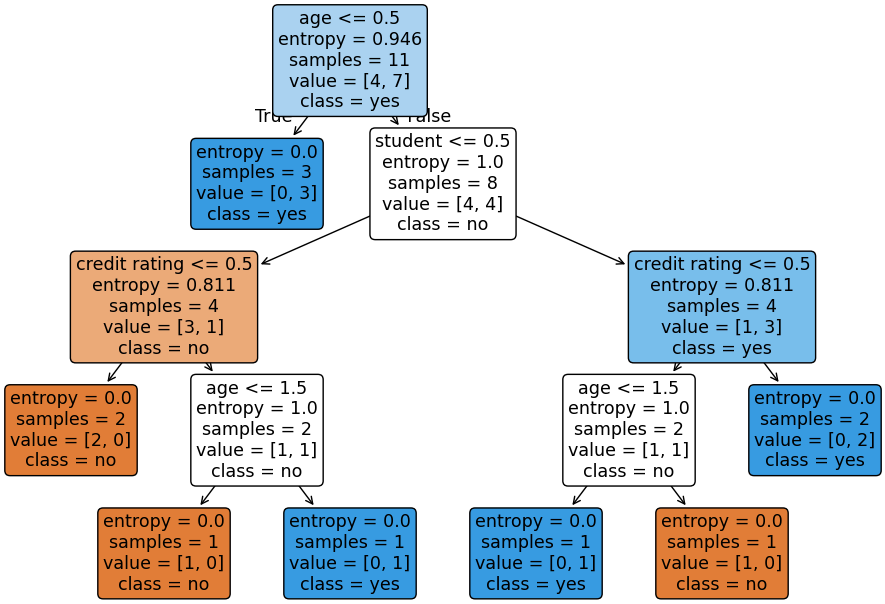
And evaluate our model.

In [68]:
```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy: 1.00
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         2

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3

Confusion Matrix:
[[1 0]
 [0 2]]
```

And visualize our tree!

In [69]:
```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

# Plot the decision tree
plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()
```

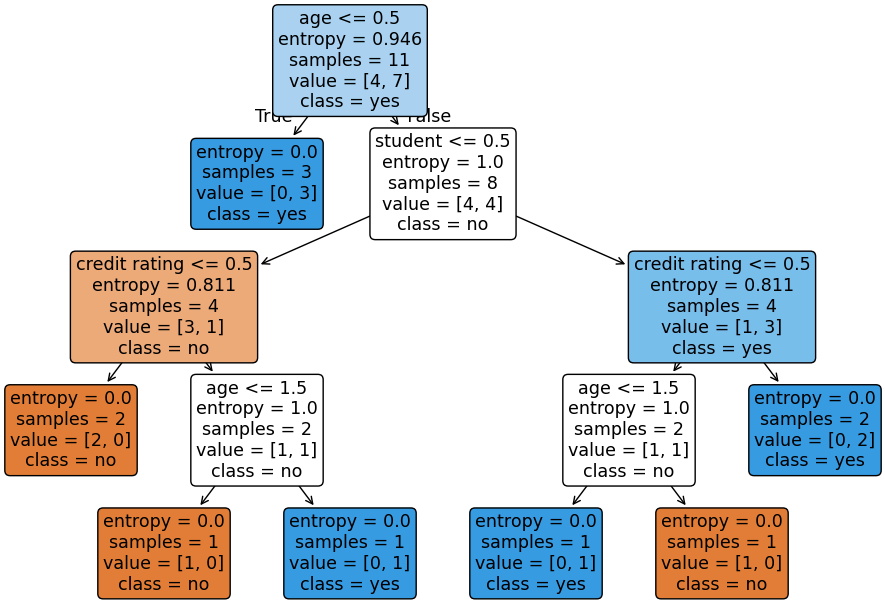


Put them all together.

In [70]:
```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

# data
data = pd.read_csv('toy_data.csv')

df = pd.DataFrame(data)

# Encode categorical columns using LabelEncoder
label_encoder = LabelEncoder()
df['age'] = label_encoder.fit_transform(df['age'])
df['income'] = label_encoder.fit_transform(df['income'])
df['student'] = label_encoder.fit_transform(df['student'])
df['credit rating'] = label_encoder.fit_transform(df['credit rating'])
df['buys computer'] = label_encoder.fit_transform(df['buys computer'])

# Separate features (X) and target (y)
X = df.drop('buys computer', axis=1)
y = df['buys computer']

# Split the dataset into training and test sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize the Decision Tree classifier
clf = DecisionTreeClassifier(criterion='entropy', random_state=42)

# Train the model
clf.fit(X_train, y_train)

# Predict on the test set
y_pred = clf.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix
print("Confusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))

# Plot the decision tree
plt.figure(figsize=(12, 8))
plot_tree(clf, filled=True, feature_names=X.columns, class_names=['no', 'yes'], rounded=True)
plt.show()
```

```
Accuracy: 1.00
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         2

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3

Confusion Matrix:
[[1 0]
 [0 2]]
```



Is the output tree the same as what you calculated yourself? Explain in your own words why they are the same or different.

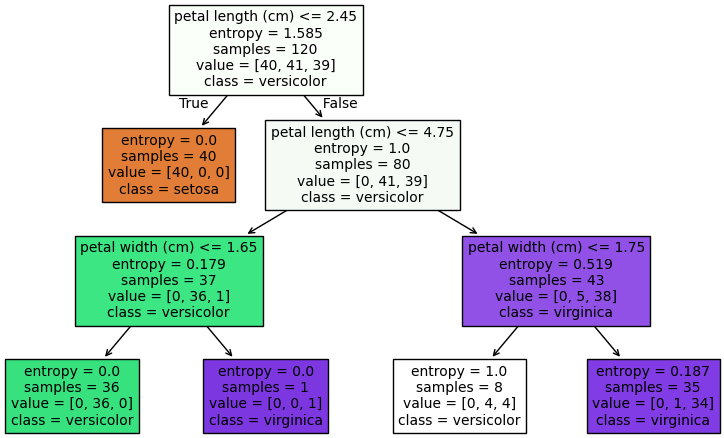
Ans: output ไม่ตรงกับที่คำนวณผ่าน function เนื่องจาก model decision tree นี้มีการแบ่ง train data 80% และ test data 20% เมื่อเราป่า train data ไปคำนวณค่า entropy ทำให้ค่าที่ได้มีความแตกต่างกัน และหากลอง train model ใหม่อีกครั้งผลลัพธ์ก็อาจจะแตกต่างไปจากเดิมเนื่องจากการสุ่ม train data และ test data

---

Another example, another dataset -- Iris

In [72]:
```
from sklearn.datasets import load_iris

# 1. Load the Iris dataset
iris = load_iris()
X = iris.data  # Features
y = iris.target  # Target labels

# 2. Split the dataset into training (80%) and testing (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Create and train a Decision Tree model with entropy criterion
clf = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
clf.fit(X_train, y_train)

# 4. Make predictions on the test set
y_pred = clf.predict(X_test)

# 5. Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.2f}")

# 6. Visualize the Decision Tree
plt.figure(figsize=(10, 6))
plot_tree(clf, filled=True, feature_names=iris.feature_names, class_names=iris.target_names)
plt.show()
```

```
Model Accuracy: 1.00
```


petal length (cm) <= 2.45
entropy = 1.585
samples = 120
value = [40, 41, 39]
class = versicolor
True
False
entropy = 0.0
samples = 40
value = [40, 0, 0]
class = setosa
petal length (cm) <= 4.75
entropy = 1.0
samples = 80
value = [0, 41, 39]
class = versicolor
petal width (cm) <= 1.65
entropy = 0.179
samples = 37
value = [0, 36, 1]
class = versicolor
petal width (cm) <= 1.75
entropy = 0.519
samples = 43
value = [0, 5, 38]
class = virginica
entropy = 0.0
samples = 36
value = [0, 36, 0]
class = versicolor
entropy = 0.0
samples = 1
value = [0, 0, 1]
class = virginica
entropy = 1.0
samples = 8
value = [0, 4, 4]
class = versicolor
entropy = 0.187
samples = 35
value = [0, 1, 34]
class = virginica

In [ ]: